

Lista de Exercícios - Cálculo Numérico

Victor Silva de Azevedo

Prof. Rafael Beserra

Sumário

Questão 2.2	2
Resposta	2
Questão 2.4 py	4
Resposta	4
Questão 3.5	6
Resposta	6
Questão 3.6	7
Resposta	7
Questão 3.7	8
Resposta	8
Questão 3.8	9
Resposta	9
Questão 4.3	10
Resposta	10
Questão 4.4	14
Resposta	14
Questão 5.9	16
Resposta	16
Questão 5.10	16
Resposta	16

Questão 2.2

Uma operação bastante comum em processamento de sinais é a convolução.

- Crie uma função $f(x) = \sin(x) + \eta(x)$ onde $\eta(x)$ é uma função de ruído branco gaussiano. Utilize `np.random.normal` com média 0 e desvio padrão pequeno, por exemplo, 0.1.
- Calcule a convolução de $f(x)$ com um filtro de média. Utilize a função `np.convolve`. O primeiro argumento dessa função é o vetor no qual quer aplicar a convolução. O segundo argumento é o chamado kernel. Escolha `np.ones(s)/s`. Quanto maior o valor de s , maior será a suavização.
- Plote em uma única figura:
 - a função $\sin(x)$;
 - a função $f(x)$;
 - o resultado da convolução;
 - o erro absoluto entre o resultado da convolução e $\sin(x)$.

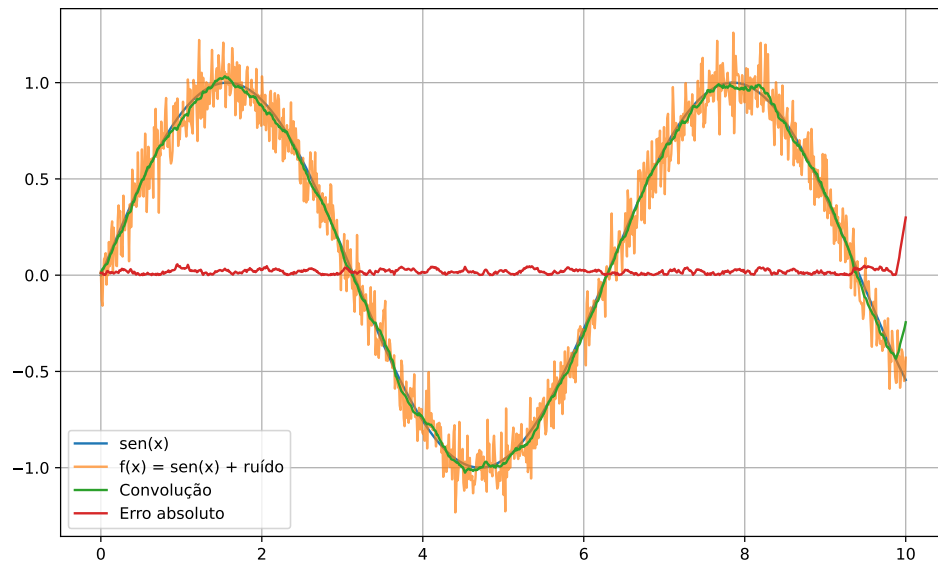
Resposta

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 1000)
seno = np.sin(x)
ruído = np.random.normal(0, 0.1, len(x))
f = seno + ruído

s = 25
kernel = np.ones(s) / s
conv = np.convolve(f, kernel, mode='same')
erro = np.abs(conv - seno)

plt.figure(figsize=(10, 6))
plt.plot(x, seno, label="sen(x)")
plt.plot(x, f, label="f(x) = sen(x) + ruído", alpha=0.7)
plt.plot(x, conv, label="Convolução")
plt.plot(x, erro, label="Erro absoluto")
plt.grid(True)
plt.legend()
plt.show()
```



Questão 2.4 py

Em algumas aplicações, uma função não é conhecida, mas sua derivada e algum ponto da função são conhecidos.

Dados:

- $f(x) = x \cos(x) + 1$ (apenas para comparação),
- $f(0) = 1$,
- $f'(x) = \cos(x) - x \sin(x)$.

Partindo de $f(0)$, estime outros pontos de $f(x)$ no intervalo $[0, 6]$ pelo método de Euler. Plote em um único gráfico:

- a função $f(x)$, - os pontos estimados, conectando-os com segmentos de reta.

Resposta

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x * np.cos(x) + 1

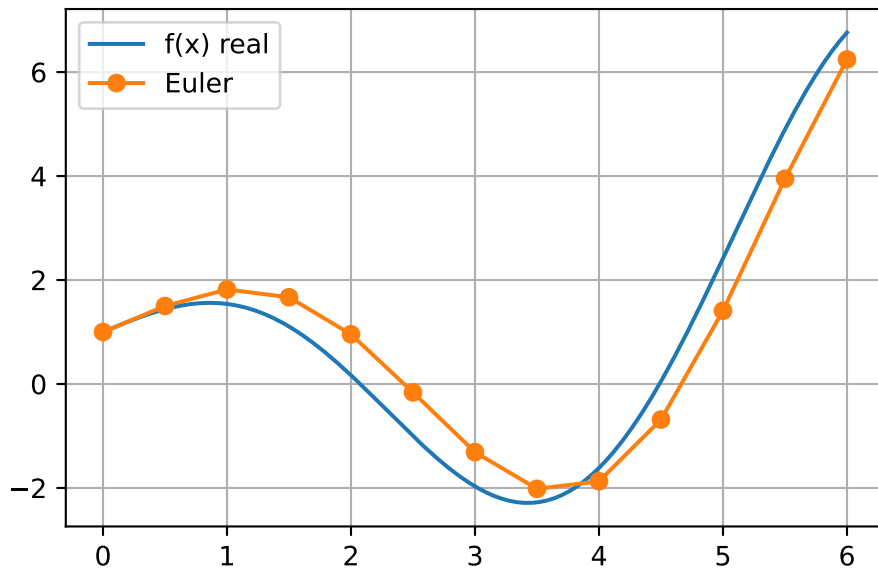
def df(x):
    return np.cos(x) - x * np.sin(x)

h = 0.5
x_aprox = np.arange(0, 6 + h, h)
y_aprox = np.zeros(len(x_aprox))
y_aprox[0] = 1

for i in range(len(x_aprox) - 1):
    y_aprox[i+1] = y_aprox[i] + h * df(x_aprox[i])

x = np.linspace(0, 6, 400)

plt.plot(x, f(x), label="f(x) real")
plt.plot(x_aprox, y_aprox, 'o-', label="Euler")
plt.grid(True)
plt.legend()
plt.show()
```



Questão 3.5

Realize as seguintes conversões de base:

a. $6201_7 \rightarrow x_3$

b. $EBA_{16} \rightarrow x_2$

c. $111000001100010111101_2 \rightarrow x_{16}$

Resposta

3.5. a) $6201_7 \rightarrow x_3$

$$6 \cdot 7^3 + 2 \cdot 7^2 + 0 \cdot 7^1 + 1 \cdot 7^0 = 2157_{10}$$

$2157 \div 3 = 719$	0
$719 \div 3 = 239$	2
$239 \div 3 = 79$	2
$79 \div 3 = 26$	1
$26 \div 3 = 8$	2
$8 \div 3 = 2$	2
$2 \div 3 = 0$	2

b) $EBA_{16} \rightarrow x_2$

$E=14, B=11, A=10$

$1110 \quad 1011 \quad 1010 \rightarrow 111010111010_2$

c) $(00011100000110001011101)_2 \rightarrow x_{16}$

$1_{10} \quad 12_{10} \quad 1_{10} \quad 8_{10} \quad 11_{10} \quad 13_{10}$

$1_{16} \quad C_{16} \quad 1_{16} \quad 8_{16} \quad B_{16} \quad D_{16} \rightarrow (1C18BD)_{16}$

Questão 3.6

Represente o número -99 em inteiro de 8 bits utilizando (1) sinal-magnitude, (2) complemento de 1 e (3) complemento de 2.

Resposta

3.6. (-99) ₁₀ em 8 bits; (-99) ₁₀ → (01100011) ₂	99 ÷ 2 = 49	1
	49 ÷ 2 = 24	1
(1) sinal-magnitude: (11100011)	24 ÷ 2 = 12	0
	12 ÷ 2 = 6	0
(2) complemento de 1: (10011100)	6 ÷ 2 = 3	0
	3 ÷ 2 = 1	1
(3) complemento de 2: (10011101)	1 ÷ 2 = 0	1

Questão 3.7

Represente o número -382.775 de acordo com o padrão IEEE 754 (precisão simples).

Resposta

3.7. -382,775 IEEE 754				0.775 × 2 = 1.55	1	382 ÷ 2 = 191	0	↑
-382,775 - 2 ⁸				0.55 × 2 = 1.1	1	191 ÷ 2 = 95	1	
-(101111110,11000110...)₂				0.1 × 2 = 0.2	0	95 ÷ 2 = 47	1	
→ Normalizar → ±1,1 × 2 ^e				0.2 × 2 = 0.4	0	47 ÷ 2 = 23	1	
↳ -1,011111101100011... × 2 ⁸				0.4 × 2 = 0.8	0	23 ÷ 2 = 11	1	
127 + e = 135 ₁₀ → 10000111 ₂				0.8 × 2 = 1.6	1	11 ÷ 2 = 5	1	
				0.6 × 2 = 1.2	1	5 ÷ 2 = 2	1	
S e(8bits) f(23bits)				0.2 × 2 = 0.4	0	2 ÷ 2 = 1	0	
1 1000 0111 011111101100011001100110				0.4 × 2 = 0.8	0	1 ÷ 2 = 0	1	
				0.8 × 2 = 1.6	1			
				0.6 × 2 = 1.2	1			

3.8. LaTeX

Questão 3.8

Você está escrevendo um programa que registra em um arquivo os comandos de um controle de Playstation. Esse controle possui um total de 14 botões, além do analógico. Mas para os registros, você considerará somente os seguintes 12 botões: $\triangle$, $\times$, $\square$, $\circ$, $\rightarrow$, $\leftarrow$, $\uparrow, \downarrow$, **LT**, **LB**, **RT**, **RB**.

- Como você poderia representar uma sequência de comandos como inteiros? Exemplifique.
- Como você poderia gravar/recuperar uma sequência de comandos de forma a economizar o máximo possível de espaço no arquivo?

Resposta

a) Podemos representar cada comando do controle um inteiro de 12 bits, em que cada bit indica o estado de um dos 12 botões. Se o bit vale **1**, o botão está pressionado, e **0** quando o botão não está pressionado. Associando os botões aos bits de 0 a 11:

bit 0 = $\triangle$
bit 1 = $\times$
bit 2 = $\square$
bit 3 = $\circ$
bit 4 = $\rightarrow$
bit 5 = $\leftarrow$
bit 6 = $\uparrow$
bit 7 = $\downarrow$
bit 8 = **LT**
bit 9 = **LB**
bit 10 = **RT**
bit 11 = **RB**

Podemos representar cada comando na forma de uma soma de potências de 2, onde cada potência corresponde a um botão:

$$Comando = \sum_{i=0}^{11} b_i 2^i, \quad b_i \in \{0, 1\}.$$

Por exemplo, se os botões $\times$, $\uparrow$ e **RT** estiverem pressionados, então os bits 1, 6 e 10 valem **1**. Logo,

$$Comando = 2^1 + 2^6 + 2^{10} = 2 + 64 + 1024 = 1090.$$

Que convertendo para binário, obtemos:

$$010001000010_2.$$

b) Para economizar espaço no arquivo, o mais adequado é gravar os comandos em formato binário. Na recuperação, basta ler cada número e verificar quais posições estão marcadas com **1**, descobrindo assim quais botões estavam pressionados.

Questão 4.3

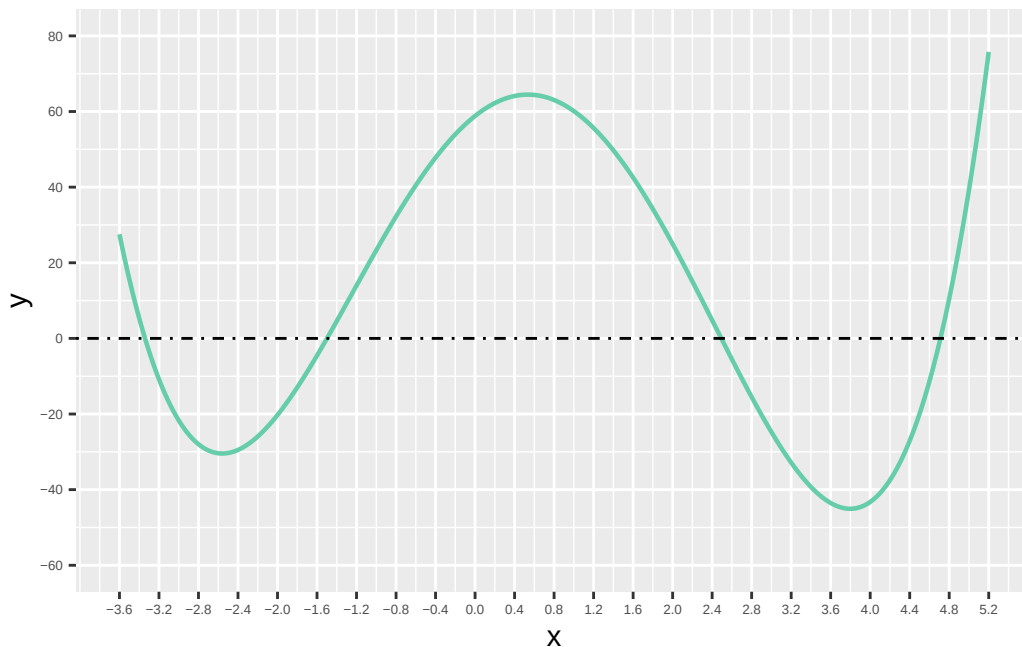
Encontre computacionalmente as 4 raízes reais de:

$$f(x) = x^4 - 2.36343x^3 - 18.1163x^2 + 20.7595x + 58.8273$$

- Métodos
 - a. Bisseção
 - b. Newton-Raphson
 - c. Secante
 - d. Regula Falsi
- na resolução, compare os métodos em relação ao número de iterações
- Implementar utilizando os templates (em C/python/R) disponíveis no github
- Metodologia:
 1. Você pode utilizar o gráfico para ter uma boa noção do intervalo inicial no caso de (a), x_0 no caso de (b) e x_0, x_1 no caso de (c) e (d).
 2. O critério de parada deve ser:
 - a. $|f(x_i)| < 0.001$ (note a mudança em relação é questão anterior)
 3. Basta que cada método encontre uma das raízes, desde que todas as 4 raízes sejam encontradas
 4. A função que implementa o método deve escrever todas as aproximações na tela (uma por linha)

Resposta

- Gráfico da função



Pelo gráfico da função, utilizei os seguintes valores para cada método:

- Bisseção: $[a, b] = [-3.5, -3.0]$
- Newton-Raphson: $x_0 = -1.6$
- Secante: $x_0 = 2.0$ e $x_1 = 3.0$

- Regula Falsi: $[a, b] = [4.5, 5.0]$

Implementação em R

```
f <- function(x) {
  x^4 - 2.36343*x^3 - 18.1163*x^2 + 20.7595*x + 58.8273
}
# derivada da funcao f(x), p/ usar no método de Newton
f1 <- function(x) {
  4*x^3 - 3*2.36343*x^2 - 2*18.1163*x + 20.7595
}

eps <- 0.001

bissecao <- function(a, b) {
  # k conta o numero de iteracoes
  k <- 0

  while (TRUE) {
    # ponto medio do intervalo atual
    x <- (a + b) / 2

    # Atualiza o contador de iteracoes
    k <- k + 1

    # Imprime a aproximacao atual
    cat(x, "\n")

    # criterio de parada
    if (abs(f(x)) < eps) {
      return(c(raiz = x, iteracoes = k))
    }

    # verifica em qual metade do intervalo tem mudanca de sinal
    # e atualiza o intervalo
    if (f(a) * f(x) < 0) {
      b <- x
    } else {
      a <- x
    }
  }
}

newton <- function(x0) {
  # k iterações
  k <- 0
  x <- x0

  while (TRUE) {
    cat(x, "\n")

    k <- k + 1

    # teste criterio de parada
    if (abs(f(x)) < eps) {
      return(c(raiz = x, iteracoes = k))
    }
  }
}
```

```

    #  $x_{k+1} = x_k - f(x_k)/f'(x_k)$ 
    x <- x - f(x) / f1(x)
  }
}

secante <- function(x0, x1) {
  k <- 0

  while (TRUE) {
    cat(x1, "\n")
    k <- k + 1

    # criterio de parada
    if (abs(f(x1)) < eps) {
      return(c(raiz = x1, iteracoes = k))
    }
    # formula
    x2 <- x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))

    # atualiza as duas ultimas aproximacoes
    x0 <- x1
    x1 <- x2
  }
}

regulaFalsi <- function(a, b) {
  k <- 0

  while (TRUE) {
    # formula falsa posição
    x <- (a * f(b) - b * f(a)) / (f(b) - f(a))

    k <- k + 1

    cat(x, "\n")
    # criterio de parada
    if (abs(f(x)) < eps) {
      return(c(raiz = x, iteracoes = k))
    }
    # mantem o intervalo [a,b] com mudanca de sinal
    if (f(a) * f(x) < 0) {
      b <- x
    } else {
      a <- x
    }
  }
}

```

- Aplicação dos métodos

```

cat("Bissecao:\n")
r1 <- bissecao(-3.5, -3)
print(r1)

cat("\nNewton:\n")
r2 <- newton(-1.7)
print(r2)

```

```
cat("\nSecante:\n")
r3 <- secante(2, 3)
print(r3)

cat("\nRegula Falsi:\n")
r4 <- regulaFalsi(4.5, 5)
print(r4)
```

- Comparação dos métodos

```
resultado <- data.frame(
  metodo = c("Bissecão", "Newton", "Secante", "Regula Falsi"),
  raiz = c(r1[1], r2[1], r3[1], r4[1]),
  iteracoes = c(r1[2], r2[2], r3[2], r4[2])
)
```

metodo	raiz	iteracoes
Bissecão	-3.341156	14
Newton	-1.499233	3
Secante	2.492896	4
Regula Falsi	4.710927	7

Questão 4.4

(Burden, pág. 95) Duas escadas se cruzam em um beco de largura L . Cada escada tem uma extremidade apoiada na base de uma parede e a outra extremidade apoia em algum ponto na parede oposta. As escadas se cruzam a uma altura A acima do pavimento. Calcule L , sendo $x_1 = 20m$ e $x_2 = 30m$ os respectivos comprimentos das escadas e $A = 8m$. Use um dos métodos para encontrar raízes utilizando como critério de parada $|f(x_i)| < 0.001$

Resposta

Questão 4.4

$x_1 = 20m$
 $x_2 = 30m$
 $A = 8$

$h_2 = \sqrt{20^2 - L^2} = \sqrt{400 - L^2}$
 $h_1 = \sqrt{30^2 - L^2} = \sqrt{900 - L^2}$

$m = \frac{y_2 - y_1}{x_2 - x_1}$
 $y = mx + b$
 $(x_1, y_1) = (0, h_1) \rightarrow m = -\frac{h_1}{L}$
 $(x_2, y_2) = (L, 0)$
 $y = -\frac{h_1}{L}x + b \rightarrow (0, h_1) \rightarrow h_1 = -\frac{h_1}{L} \cdot 0 + b \rightarrow b = h_1$
 $x_1 \rightarrow y = h_1 - \frac{h_1}{L}x$

$(x_1, y_1) = (0, 0) \rightarrow m = \frac{h_2}{L}$
 $(x_2, y_2) = (L, h_2)$
 $y = \frac{h_2}{L}x + b \rightarrow (0, 0) \rightarrow 0 = b \rightarrow y = \frac{h_2}{L}x$

Cruzamento das rotas: $y = y$
 $\frac{h_2}{L}x = h_1 - \frac{h_1}{L}x$
 $h_2x = Lh_1 - h_1x$
 $xh_2 + xh_1 = Lh_1$
 $x(h_2 + h_1) = Lh_1$
 $x = \frac{Lh_1}{h_2 + h_1}$

Substituindo em y :
 $y = \frac{h_2}{L} \left(\frac{Lh_1}{h_2 + h_1} \right) = \frac{h_2h_1}{h_2 + h_1} = A$
 $A = \frac{h_2h_1}{h_2 + h_1}$
 $f(L) = \frac{h_2h_1}{h_2 + h_1} - 8$

L	$f(L)$	$[a, b] = [16, 17]$
15	0,8157	Vou usar o método
16	0,1474	Regula Falsi
17	-0,6329	$x_i = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$

```
eps <- 0.001
```

```
f <- function(L) {
  (sqrt(400 - L^2) * sqrt(900 - L^2)) /
  (sqrt(400 - L^2) + sqrt(900 - L^2)) - 8
}
```

```
regulaFalsi <- function(a, b) {
  k <- 0

  while (TRUE) {
    # formula falsa posição
    x <- (a * f(b) - b * f(a)) / (f(b) - f(a))
```

```

    k <- k + 1

    cat(x, "\n")
    # criterio de parada
    if (abs(f(x)) < eps) {
        return(c(raiz = x, iteracoes = k))
    }
    # mantem o intervalo [a,b] com mudanca de sinal
    if (f(a) * f(x) < 0) {
        b <- x
    } else {
        a <- x
    }
}
}

r4 <- regulaFalsi(16, 17)

```

```

16.19384
16.21054
16.21199

```

```
print(r4)
```

```

      raiz iteracoes
16.21199    3.00000

```

```

resultado <- data.frame(
  raiz = r4[1],
  iteracoes = r4[2]
)

```

	raiz	iteracoes
raiz	16.21199	3

Questão 5.9

Calcule a matriz inversa de:

$$A = \begin{bmatrix} 1 & 3 & 4 \\ -2 & -5 & -6 \\ 3 & 8 & 11 \end{bmatrix}$$

Resposta

```
A <- matrix(c(
  1, 3, 4,
  -2, -5, -6,
  3, 8, 11
), nrow = 3, byrow = TRUE)
A_inv <- solve(A)
A_inv
```

```
      [,1] [,2] [,3]
[1,]   -7   -1    2
[2,]    4   -1   -2
[3,]   -1    1    1
```

Questão 5.10

Implemente o algoritmo de eliminação de Gauss-Jordan utilizando os templates (em C ou python) disponíveis no github. Teste com os sistemas lineares disponibilizados pelo professor no repositório. O arquivo de entrada contém um inteiro n , seguido de n linhas com $n + 1$ números cada (a linha i contém cada a_{ij} , seguidos de b_i). A leitura dessa matriz já é realizada no template. Aplique uma iteração de refinamento e analise o impacto na norma do vetor resíduo.

Resposta
